Week 1 – Research Design and Toolchain Kickoff

Physics-informed AI for Three-phase Unbalanced N-1 Security Screening in Microgrids

Course Instructor

Microgrid 3-phase Security AI Project

Week 1

Keywords: microgrid, three-phase unbalanced, N-1 screening, physics-informed AI, GNN, pandapower

What this lecture covers

Part A – How this course works

- Textbook learning vs research learning
- Working backward from the paper
- Mindset shifts you will need

Part B – Research design

- Why microgrid security is changing
- N-1 screening as a research question
- Working hypothesis
- Eight-week roadmap

Part C – Knowledge stack

- Power systems prerequisites
- Machine learning prerequisites
- Recommended reading

Part D – Toolchain setup

- Installing VSCode + micromamba
- Creating the `pdpower` environment
- First notebook smoke test

Goal of Week 1

By the end of this session, every student should have (a) a clear picture of how this course differs from a typical lecture course, (b) a working development environment, and (c) a mental model of what we are trying to prove over eight weeks.

Two modes of learning

	Textbook learning	**Research-style learning**
Direction	Forward: definitions → exercises	Backward: deliverable → what we need to learn
Problems	Closed, a clean answer exists	Open, no oracle anywhere
Authority	Instructor holds the answer key	No one has the full answer; you become the local expert
Knowledge	Acquire first, apply later	Acquire because the project needs it now
Failure	Getting a "wrong" answer	A negative result is data, not failure
Goal	Master a fixed body of material	Produce something that did not exist before
Assessment	Exam testing recall	Reproducibility, validation, defense of choices

Where this course sits

Most undergrad courses are textbook-mode. This one is research-mode *applied* to a teaching format. The skills it builds — defining your own question and validating your own work — are the skills you will need for a thesis and for the first years of a research job.

Project-driven: working backward from the paper

The mini-paper at Week 8 is the destination. We work backward to figure out what each earlier week must produce:

Week 8 – Paper	needs three model families compared under a screening metric
↓	*requires*
Week 7 – GNN	needs a graph dataset and ML baselines to beat
↓	*requires*
Week 6 – PI-MLP	needs labeled samples plus a physics-loss design
↓	*requires*
Week 5 – Baselines	needs a clean, leakage-free feature table
↓	*requires*
Week 4 – N-1 labels	needs many operating scenarios to scan
↓	*requires*
Week 3 – Scenarios	needs a working three-phase PF engine
↓	*requires*
Week 2 – 3-phase PF	needs an environment and a research question
↓	*requires*
Week 1 – Today	alignment on what "success at Week 8" means

Implication for how you study

Each notebook is a *tool we need to build the next thing*, not a chapter to memorize. If you ever wonder

Five mindset shifts you will need

1. **Own your piece.** You will become the local expert on a sub-question nobody else has run before. The instructor is your collaborator, not your answer key.
2. **Learn just-in-time, not just-in-case.** Do not read the whole pandapower manual before Week 2. Read the section you need when you need it. The project pulls the knowledge.
3. **Validation beats vibes.** "My code ran" is not evidence of correctness. Every result needs a proof, a cross-check, or an audit. Roughly 20% of each week is spent here.
4. **Negative results are data.** When the phase-aware GNN underperforms a flat MLP on 77 samples, that is a finding to document, not a failure to hide.
5. **"I don't know yet" is the senior move.** Saying "I don't know yet, but here is how I will find out" is more respected than guessing confidently. It is what every working researcher says ten times a day.

Why the grid is harder than ten years ago

- Distributed PV, battery storage, EV charging are now everywhere on the LV side.
- Many loads and inverters are **single-phase**, so the three phases of a feeder no longer carry the same power.
- Microgrids can run *grid-connected* or *islanded*; the same fault can have very different consequences.
- Operators must decide quickly: *is this state still safe if one component trips?*

Consequence

A balanced positive-sequence power flow can declare a feeder safe while one phase is actually undervoltage or overloaded. We need phase-aware analysis.

Static N-1 security, in one slide

N-0 (base case)

All planned equipment is in service. We check that the power flow converges and that voltages and loadings are inside limits.

N-1 (contingency)

Any single critical element trips: a line, a transformer, the PCC tie, a DER, or the battery. We check whether the post-contingency state is still safe.

Violation types we will label

- Phase under/over voltage
- Line or transformer overload
- Voltage unbalance factor (VUF)
- Service loss (loads disconnected)
- Non-convergence (no feasible solution)

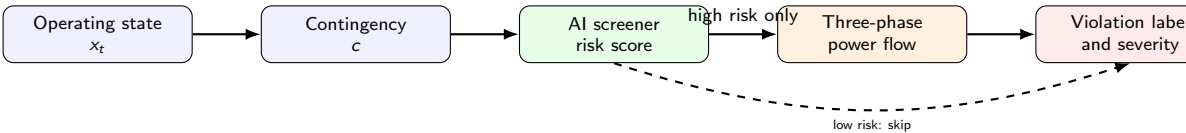
Why exhaustive N-1 is expensive

- For each operating point x_t and each contingency c, we run a three-phase power flow $f_{3\phi}(x_t, c)$.
- Realistic studies sweep many scenarios: PV variability, load uncertainty, EV charging, BESS dispatch, daily cycles.
- Contingency catalog: line outages, transformer outages, PCC outage, DER trip, BESS trip, switch operations.
- Total cost grows as $|\text{scenarios}| \times |\text{contingencies}|$.

Opportunity

Most contingencies do *not* cause violations. If a fast model can prune the safe ones with high recall, three-phase power flow only needs to be re-run on the suspicious ones.

The screening idea



Design principle

The AI model does not replace the power flow. It *prioritizes* which contingencies are worth re-solving. We optimize for **recall**, not accuracy.

Research question and hypothesis

Research question

Can a physics-informed, phase-aware AI model maintain very high recall on three-phase N-1 violations while reducing the number of full `runpp_3ph` calls?

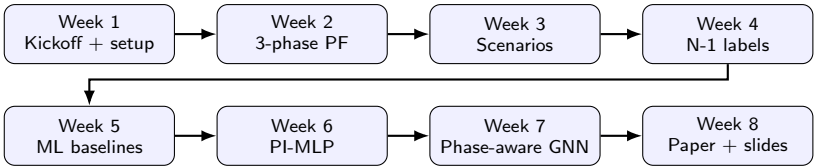
Working hypothesis

If the model jointly uses (i) microgrid topology, (ii) per-phase quantities, (iii) contingency masks, and (iv) physical limit constraints, then it can keep violation recall close to 1.0 while saving 30–60% of power-flow calls.

What we are *not* claiming

We are **not** replacing power-flow software, not solving optimal power flow, and not making deployment-grade claims on a small teaching feeder.

Eight-week roadmap



Deliverables at the end

A mini-paper draft (IEEEtran), a 15-slide defense deck, and a validation suite that proves each result.

What you will need to learn (1/2)

Power systems

- Three-phase circuits, per-unit, Δ/Y connections
- Symmetrical components: positive, negative, zero sequence
- Voltage unbalance factor (VUF)
- AC power flow: Newton-Raphson and backward/forward sweep
- Three-phase / asymmetric power flow
- N-1 contingency analysis

Microgrid engineering

- PCC, DER, BESS, critical load
- Grid-connected vs islanded modes
- Inverter modeling at a high level
- Service-loss and topology connectivity

Honest expectation

You do not need to be an expert before Week 1. You will learn most of these as we go. But please review three-phase basics and per-unit before Week 2.

What you will need to learn (2/2)

Machine learning

- Classification, regression, ranking
- Train / validation / test splits, group-based CV
- Class imbalance, recall, FNR, AUPRC
- MLPs and basic PyTorch training loops
- Graph Neural Networks (message passing)
- Physics-informed loss design

Software / engineering

- Python 3, NumPy, pandas, matplotlib
- Jupyter notebooks (and how not to abuse them)
- VSCode as an IDE
- Conda / mamba environments
- Reading and using pandapower documentation

Recommended reading order

1. **pandapower documentation** – start with *Getting Started*, then *AC Power Flow*, then *Asymmetric / Three-phase Power Flow*.
2. **Grainger & Stevenson**, *Power System Analysis* – chapters on three-phase circuits and symmetrical components.
3. **Goodfellow et al.**, *Deep Learning* – chapter 6 (MLPs) and chapter 7 (regularization).
4. **Hamilton**, *Graph Representation Learning* (free online) – chapters 1–5 cover everything we need for Week 7.
5. **Raissi et al.**, *Physics-informed Neural Networks* (JCP 2019) – the canonical reference for PINN ideas.

How to use this list

You are not expected to finish all of these before Week 2. Treat them as a reference you return to when a concept comes up in lecture.

The setup we are about to do

Why this stack

VSCode + mamba is reproducible across macOS, Linux, and Windows, supports Jupyter inline, and avoids the dependency hell of mixing system Python with pip.

Step 1: Install VSCode

1. Go to `https://code.visualstudio.com/` and download the installer for your OS.
2. Run the installer with default options.
3. Launch VSCode and sign in (optional) for settings sync.
4. Open the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`) and install:
   - **Python** (Microsoft)
   - **Jupyter** (Microsoft)
   - **Pylance** (Microsoft)

Why VSCode

- Native Jupyter notebook UI
- Same UI on macOS, Linux, and Windows
- Strong Python type-checking via Pylance
- Lightweight compared to a full IDE

Step 2: Install micromamba (macOS / Linux)

Why micromamba, not Anaconda?

- Single static binary, fast solver (libsolv), no large base environment.
- Works exactly the same on macOS, Linux, and inside Docker.

macOS / Linux (recommended one-liner):

```
"${SHELL}" <(curl -L micro.mamba.pm/install.sh)
```

Follow the prompts. The installer adds a `micromamba` entry to your shell config (usually `~/.zshrc` or `~/.bashrc`). Restart the shell after install. **Verify:**

```
micromamba --version
# 2.x or higher
```

Step 2 (alt): Install on Windows

Option A – Windows PowerShell (native):

```
Invoke-Expression ((Invoke-WebRequest -Uri https://micro.mamba.pm/install.ps1 -UseBasicParsing).Content)
```

Option B – WSL (recommended for this course):

1. Install WSL 2 with Ubuntu from the Microsoft Store.
2. Open the Ubuntu shell and run the macOS/Linux one-liner from the previous slide.
3. In VSCode, install the **WSL** extension. Open notebooks from inside WSL.

Recommendation

WSL gives a consistent experience with the macOS/Linux instructions used throughout the course. All commands shown later assume a bash/zsh shell.

Step 3: Create the pdpower environment

Run the following *one* command in your terminal:

```
micromamba create -n pdpower -c conda-forge -y \
  python=3.11 "pandapower>=3.2" "numpy<2" \
  "pandas<2.3" "scipy<1.14" scikit-learn \
  matplotlib networkx numba pillow pytorch \
  jupyter ipykernel nbconvert
```

This installs everything we need for the eight weeks:

- `pandapower>=3.2` – provides `runpp_3ph` for unbalanced PF.
- `numpy<2`, `pandas<2.3`, `scipy<1.14` – pinned for pandapower compatibility.
- `pytorch` – CPU build is fine for this course.
- `scikit-learn`, `matplotlib`, `networkx`, `numba`, `pillow`.
- `jupyter`, `ipykernel`, `nbconvert` – so VSCode can run notebooks.

First run downloads ∼1.5 GB and takes 5–10 minutes.

Step 4: Activate and verify

```
micromamba activate pdpower
python --version # Python 3.11.x
python -c "import pandapower as pp; print(pp.__version__)"
# 3.2.1 or higher
python -c "import torch; print(torch.__version__)"
# 2.x
```

Quick three-phase power-flow check:

```
python - <<'PY'
import pandapower as pp
from pandapower.networks import ieee_european_lv_asymmetric
net = ieee_european_lv_asymmetric()
pp.runpp_3ph(net)
print("converged:", net.converged, "buses:", len(net.bus))
PY
```

You should see `converged: True buses: 907`.

Step 5: Open the shared notebook in VSCode

1. Save the `.ipynb` file your instructor shared into a folder of your choice.
2. Open VSCode. Use `File` → `Open File...` and select the notebook.
3. Open the command palette (`Cmd/Ctrl+Shift+P`) and run **Python: Select Interpreter**.
4. Choose the interpreter under `.../envs/pdpower/bin/python`.
5. Click **Run All** at the top of the notebook.

Smoke-test pass criteria

All code cells execute with no error output. If a cell turns red, copy the traceback and bring it to office hours.

Optional: headless smoke test from the terminal

If you want to verify a notebook without opening VSCode:

```
micromamba run -n pdpower jupyter nbconvert \
  --to notebook --execute path/to/your_notebook.ipynb \
  --output /tmp/smoketest.ipynb \
  --ExecutePreprocessor.timeout=600
```

Expected output (no traceback):

```
[NbConvertApp] Converting notebook ...
[NbConvertApp] Writing <N> bytes to /tmp/smoketest.ipynb
```

If you see this, your environment can execute the course notebooks end-to-end.

Common pitfalls during setup

1. △ Installing pandapower via `pip` on top of system Python. Always create and activate `pdpower` first.
2. △ Using a Python interpreter other than `pdpower` in VSCode. Always check the interpreter shown in the bottom-right corner.
3. △ Installing pandapower 2.x. The course notebooks use 3.x column names (`pl_a_mw`, `ql_a_mvar`); 2.x will crash on the first PF result cell.
4. △ Installing NumPy 2.x. pandapower still imports `np.Inf`; pin `numpy<2`.
5. △ Running notebooks on Apple Silicon with a stale x86 conda. Reinstall micromamba native to your CPU.
6. △ Forgetting to `micromamba activate pdpower` before launching Jupyter or running Python.

Week 1 homework

Required

1. Install VSCode, micromamba, and the `pdpower` environment on your laptop.
2. Open the Week 1 notebook your instructor shared and run it end-to-end with zero errors.
3. Write a 5–8 sentence paragraph: *Why is three-phase unbalanced N-1 screening interesting? What would a successful project look like at the end of Week 8?*

Optional

1. Open the IEEE European LV asymmetric network in pandapower and plot the bus topology with `networkx`.
2. Run `pp.runpp_3ph(net)` on it and inspect the per-phase voltage distribution.

Exit checklist

1. ✓ I can state the research question in one sentence.
2. ✓ I can explain why a balanced power flow is insufficient for LV microgrids.
3. ✓ I know what *recall*, *FNR*, and *PF calls saved* mean in this project.
4. ✓ VSCode is installed with the Python and Jupyter extensions.
5. ✓ `micromamba --version` works and `pdpower` is listed in `micromamba env list`.
6. ✓ `pandapower.runpp_3ph` converges on the IEEE LV network.
7. ✓ I have run the Week 1 notebook end-to-end with no errors.
8. ✓ I have a clear mental picture of the eight-week project arc.

What's next in Week 2

Week 2 will introduce

- `create_asymmetric_load`, `create_asymmetric_sgen`
- Line zero- and negative-sequence parameters
- `pp.runpp_3ph` solver internals
- Per-phase result tables: `res_bus_3ph`, `res_line_3ph`
- Voltage unbalance factor computation
- Sanity proofs at 10^{-8} tolerance

Pre-class action

Make sure your environment passes today's smoke test before Week 2. We will not debug installs during the lecture.

References and resources

- **pandapower documentation:** https://pandapower.readthedocs.io
- **micromamba install guide:** https://mamba.readthedocs.io/en/latest/installation/micromamba-installation.html
- **VSCode Python tutorial:** https://code.visualstudio.com/docs/python/python-tutorial
- **PyTorch tutorials:** https://pytorch.org/tutorials/
- **Hamilton, Graph Representation Learning** (free book): https://www.cs.mcgill.ca/~wlh/grl_book/
- **Raissi, Perdikaris, Karniadakis,** *Physics-informed Neural Networks*, JCP 2019.

Office hours

If you cannot get the environment working after one honest hour of attempts, bring your laptop and the exact error message to office hours. Do not silently struggle for a week.